

Università
degli Studi
di Padova

LECTURE NOTES
OF

MACHINE LEARNING FOR HUMAN DATA

A Collection of Notes Based on the Lectures of Prof. M. Rossi

Created By

Gabriele Bellavia

University of Padua
Academic year 2026–2027

October 13, 2026

About these notes

These notes are part of my `PoD-collection-of-notes` repository and were written during my MSc in Physics of Data at the University of Padua between 2024 and 2026.

They are intended as freely accessible study material and may contain errors, inaccuracies, or outdated information. Most of the material was written while attending the courses and was not systematically reviewed afterwards. Course contents and syllabi may also change over time.

License

Except where otherwise noted, this work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0).

You are free to share and adapt this material for non-commercial purposes, provided that appropriate credit is given, a reference to the original source is included, and any derivative work is distributed under the same license.

Source files

The complete source files used to compile these notes, including figures and other resources, are available in the corresponding folder of the repository:

`https://github.com/gabriele-bellavia/PoD-collection-of-notes`

Citation

If you use these notes, redistribute them, or use them as a starting point for your own material, please cite the original repository and, where applicable, the authors of any external material referenced in these notes.

Errors and corrections

If you find an error, typo, or unclear explanation, please feel free to open an issue on the GitHub repository or contact me at:

`gabriele.bellavia@studenti.unipd.it`

Physics of Data MSc
University of Padua

Contents

Introduction **1**
Course Overview 1

I Unsupervised learning **5**

1 Principal Component Analysis **7**
1.1 What is PCA? 7
1.1.1 Toy Example: The Spring Experiment 7
1.2 Intuition and Data Representation 8
1.2.1 Noise and Variance 9
1.2.2 Redundancy 10
1.3 Covariance and Redundancy 11
1.3.1 What Should the New Representation Look Like? 11
1.4 Maximum Variance Formulation 12
1.5 PCA through Eigendecomposition 14
1.5.1 Eigendecomposition of a Symmetric Matrix 14
1.5.2 Practical Steps 15
1.6 Dimensionality Reduction and Reconstruction 15
1.6.1 Exact Reconstruction 15
1.6.2 Approximation and Reconstruction Error 16
1.7 Summary of PCA Assumptions 17
1.8 Applications 17
1.8.1 Handwritten digits 17
1.8.2 Image Compression 19
1.9 Where PCA Fails 20
1.10 Final Remarks 20
1.11 Connection with SVD 21

A Useful Algebra Results **23**
A.1 Quadratic Forms 23
A.1.1 Definition and Symmetric Representation 23
A.1.2 Definiteness 23
A.1.3 Gradient of a Quadratic Form 24
A.1.4 Gradient of the Squared Norm 24
A.2 Eigenvalues and Eigenvectors 25
A.2.1 Linear Independence of Eigenvectors 25
A.2.2 Orthogonality for Symmetric Matrices 25

A.2.3 Eigenvalues and Definiteness 26
A.3 Convecity of Quadratic Forms 26
A.4 Covariance Matrices and Matrix Products 27
A.4.1 Positive Semidefiniteness of Covariance Matrices 27
A.4.2 Symmetry and Positive definiteness of Matrix Products 28
A.4.3 Null Spaces and Rank 28
A.5 Singular Value Decomposition Theorem 28

References **31**

Introduction

Course Overview

This course explores modern machine and deep learning techniques tailored to biological signal processing and human data analysis. The aim is to gain the theoretical foundations and practical expertise required to design, train, and validate custom models for complex biological datasets.

When talking about human data, we refer to a very broad concept. It includes physiological signals, such as blood pressure or ECG signals, as well as movement and speech activity. Mobility, vehicular, and energy consumption patterns in smart cities can also fall within the broader context of human data.

A key characteristic of many of these data is that they differ from static images or plain text commonly considered in standard machine learning settings. Human-generated data are often **dynamic** and **spatio-temporal**, showing complex correlations across both spatial dimension (e.g., multi-channel sensor arrays, EEG grids) and temporal domains (e.g., continuous physiological time series, motion dynamics). Therefore, the i.i.d. assumption will often no longer hold, since observations may be correlated in space, time, or both.

These characteristics influence the type of learning approaches that can be applied. The course will mainly focus on **unsupervised learning**, **feature extraction**, and **anomaly detection** in the context of real-world biological data.

Anomaly detection is particularly useful when abnormal events are rare and therefore difficult to collect and label in sufficiently large datasets. For example, events such as heart attacks cannot usually be observed frequently enough to build a large balanced labeled dataset. One approach is therefore to model the normal physiological state of a person and detect deviations from this normal behavior. By quantifying these deviations, it may be possible to identify potentially alarming conditions when they become sufficiently large.

The course follows an application-driven approach, with the goal of developing tools that can be applied to real-world problems across different fields. At the same time, it is research-oriented, with particular attention to current research ideas and emerging approaches. Part of the course will also focus on bio-inspired models and learning systems, namely models inspired by the way the brain processes information and learns.

The main types of signals we will deal with are:

- **Electrocardiogram (ECG):** the main application is biomedical devices, with objectives such as efficient transmission (TX) in resource-limited systems, automated anomaly detection, and user identification or authentication.

- **Speech:** we will study how to represent speech signals, how to construct an appropriate feature space, and which classifiers can be used for different tasks.

- **Inertial signals (mobility):** accelerometer and gyroscope signals obtained from chest bands, wristbands, smartwatches, smartphones, and similar devices. Typical objectives include activity recognition, user identification, and anomaly detection.
- **Electroencephalography (EEG):** a typical example of a spatio-temporal signal, where multiple sensors located on the scalp record temporal signals that are correlated with one another.

An important aspect of modern machine learning systems to keep in mind is their computational cost, both during training and inference. Table 1 shows the estimated training energy required by different generations of GPT models, highlighting the non-negligible energy cost associated with model training. However, training is only part of the computational cost and inference can also be expensive. Table 2 reports the approximate energy cost of different inference tasks, using the analysis of a single image as a reference point. Therefore, when dealing with large-scale machine learning models, computational cost should always be taken into account.

An interesting comparison can be made with biological systems and, in particular, the human brain. Suppose we want to simulate the brain as an ANN with a temporal resolution of 1 ms. Considering approximately 8.6×10^{10} neurons, around 7×10^3 synapses per neuron, and 1×10^3 updates per second, we obtain

$$\underbrace{8.6 \times 10^{10}}_{\text{neurons}} \times \underbrace{7 \times 10^3}_{\text{synapsis}} \times 1 \times 10^3 \,\text{updates/s} = 600\,\text{PetaFLOPS}^1$$

Although this is a rough estimate, it gives an idea of the computational scale involved. Nowadays, the most powerful supercomputers are capable of ~ 442 PetaFLOPS while consuming around 30 MW, whereas the human brain operates at roughly 12 W to 20 W. This huge difference suggests that biological neural systems rely on architectural and computational mechanisms that are fundamentally different from those used in conventional ANNs.

This difference motivates active research toward the so-called third generation of neural networks. One of the most promising approaches within this generation regards the so-called Spiking Neural Networks (SNNs). As the name suggests, SNNs take inspiration from the brain, where neurons communicate through discrete electrical events called **spikes**. Unlike ANNs, where information is generally represented by continuous-valued activations, in SNNs information can be encoded through the occurrence and timing of spikes. A spike can be modeled as an **all-or-nothing event**: a neuron generates a spike when its internal state reaches a certain threshold. These spikes may be triggered by sensory inputs or by the activity of other neurons in the network.

An important aspect is that activity does not need to be propagated through the entire network at every time step. Instead, computation can be **event-driven**: neurons communicate mainly when spikes occur, and only the relevant parts of the network may become active. This leads to the key concept of **sparsity**. Neural activity can be sparse both in time, since a neuron does not continuously generate spikes, and in space, since only a subset of neurons may be active at a given moment. This sparse and asynchronous computation is one of the main reasons why SNNs are considered a promising approach for energy-efficient and brain-inspired computing.

[1] One PetaFLOP corresponds to 1 quadrillion (10^{15}) floating point operations per second

Generation	Training Energy	Homes Powered (for 1 Full Year)	Gasoline Cars Driven (Lifetime Equivalent)	Round-Trip Flights (NY to London)
GPT-4 1.8 trillion params	∼50,000–62,000 MWh	∼4,800–5,900 homes	∼3,100–3,900 cars (entire lifetime)	∼15,000–18,500 passenger seats
GPT-5 3–5 trillion params	∼250,000–400,000 MWh	∼24,000–38,000 homes	∼15,500–25,000 cars	∼75,000–120,000 passenger seats
GPT-6 10+ trillion params	∼1.2–2.0 TWh	∼115,000–190,000 homes (a medium city like Salt Lake City or Florence)	∼75,000–125,000 cars	∼360,000–600,000 passenger seats

Table 1: Estimated training cost of LLMs.

Activity	Energy Consumed (kWh)	Equivalent 1-Min Audio Decodes	Equivalent Image Analyses	Equivalent 1-Min Video Analyses
Decoding 1 min of audio	0.0002 kWh	1× (Baseline)	∼0.2×	∼0.01×
Analyzing 1 image	∼0.001 kWh	∼5 audio decodes	1× (Baseline)	∼0.04×
Analyzing 1 min of video	∼0.025 kWh	∼125 audio decodes	∼25 image analyses	1× (Baseline)
Charging a smartphone (0–100%)	∼0.012 kWh	∼60 audio decodes	∼12 image analyses	∼0.5 min video
Boiling 1 cup of water (electric kettle)	∼0.030 kWh	∼150 audio decodes	∼30 image analyses	∼1.2 min video
Microwaving a frozen pizza (8–10 min)	∼0.120 kWh	∼600 audio decodes	∼120 image analyses	∼5 min video
Driving an electric car (1 km)	∼0.180 kWh	∼900 audio decodes	∼180 image analyses	∼7 min video
80 km passenger trip by train	∼2.400 kWh	∼12,000 audio decodes	∼2,400 image analyses	∼96 min video

Table 2: Energy consumption of different computational and everyday activities.

Part I

Unsupervised learning

Chapter 1

Principal Component Analysis

1.1 What is PCA?

Principal Component Analysis (PCA) is a linear, unsupervised technique used for dimensionality reduction. It finds a new set of orthogonal directions along which the variance of the data is maximized. By retaining only some of these directions, we can represent the data using fewer variables.

Given a dataset, PCA can be computed using simply matrix decomposition, without training a model through an iterative learning process. Thus, it is completely deterministic and repeating it leads basically to the same result (each principal direction is defined up to a sign). Thus, we can improperly say that it solves the problem *exactly*.

Although its simplicity, it is still widely used nowadays since it has many applications, including lossy data compression, automatic feature extraction, data visualization, and preprocessing before clustering or classification. In exploratory data analysis, PCA can help us visualize high-dimensional data and identify possible patterns or groups.

Despite its beautiful and strong mathematical foundation, PCA has some limitations in modern ML systems because it can only capture linear relationships in the data. For example, non-linear models, such as autoencoders, can learn more complex representations.

1.1.1 Toy Example: The Spring Experiment

Before going into math, let us consider a simple example to understand the context and the setup for PCA.

Consider a mass attached to an ideal spring, moving without friction along the x-axis. The spring is stretched away from its equilibrium point and released, causing the mass to oscillate at a fixed frequency. In this setup, a single variable x is sufficient to fully characterize the law of motion.

However, suppose that we do not know the underlying system and instead measure the motion using three cameras placed at different angles. Each camera observes a different projection of the same underlying motion and records two coordinates. Combining the observations gives a six-dimensional measurement vector.

This is similar to what often happens in real experiments: we may record more variables than needed and oversample the signal using a sub-optimal reference system because we do not know in advance which measurements are the most useful. This introduces a lot of redundant information where it is not needed. Citing the original paper [1],

We know a-priori that if we were smart experimenters, we would have just measured the position along the x-axis with one camera. But this is not what happens in the real world. We often do not know which measurements best reflect the dynamics of our system [...] *Furthermore, we sometimes record more dimensions than we actually need.*

The experiment setup is shown in Figure 1.1, where three cameras observe the spring motion from different angles. The point clouds below show the measurements recorded by each camera. The points do not lie exactly on straight lines because the measurements also contain noise.

The goal of PCA is to automatically find a new basis in which the dataset can be re-expressed, so that most of the variance is concentrated along a smaller number of directions, ideally making the underlying structure of the data easier to identify while reducing the contribution of less informative directions and noise. In this example, we expect one principal direction to capture most of the variability associated with the spring motion. If the remaining directions mainly describe measurement noise, retaining only this first component gives a single meaningful variable that approximately represents the motion. This makes the underlying structure easier to identify while reducing redundancy and the contribution of noise.

This allows us to keep in mind two key concepts: **informative data** and less relevant variability – e.g., **noise**.

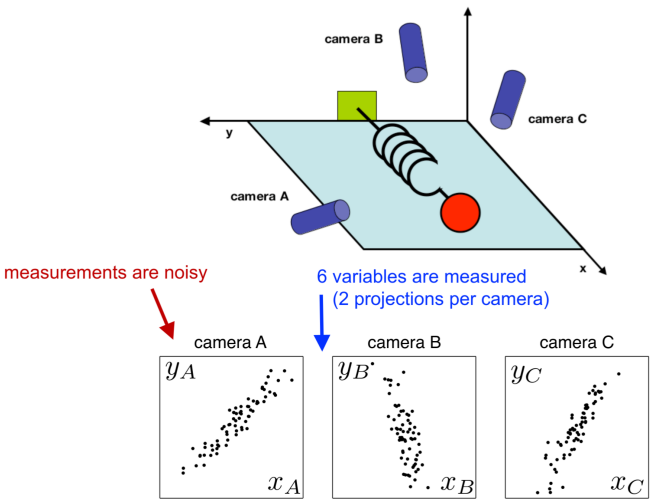


Figure 1.1

1.2 Intuition and Data Representation

Consider n samples, each containing m measurements. In the spring experiment, $m = 6$, and each sample corresponds to one observation time. We represent a sample as a *column vector*:

$$\mathbf{x}_i = (x_{1i}, x_{2i}, \ldots, x_{mi})^T \in \mathbb{R}^m.$$

The entire dataset can be collected in the matrix

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \ldots & \mathbf{x}_n \end{bmatrix} \in \mathbb{R}^{m \times n},$$

where each row corresponds to a measurement type, and each column to a sample. We assume that the data are zero-centered, otherwise we compute the sample mean and center the data:

$$\overline{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n}\mathbf{x}_i, \qquad \mathbf{x}'_i = \mathbf{x}_i - \overline{\mathbf{x}}. \tag{1.1}$$

Trivially, each vector in the dataset can be expressed as a linear combination of the vectors of the standard basis, with coefficients given by the coordinates of the data point itself:

$$\mathbf{x}_i = \mathbf{B}\mathbf{x}_i,$$

where

$$\mathbf{B} = \begin{bmatrix}\mathbf{b}_1\\ \mathbf{b}_2\\ \vdots\\ \mathbf{b}_m\end{bmatrix} = \begin{bmatrix}1 & 0 & \dots & 0\\ 0 & 1 & \dots & 0\\ \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \dots & 1\end{bmatrix}.$$

The columns of $\mathbf{B}$ are the standard basis vectors of $\mathbb{R}^m$.

The same data can also be described using a different coordinate system. We therefore ask whether another basis, whose vectors are linear combinations of the original basis, could *better represent* the structure of the dataset. PCA answers this question by making a strong but powerful assumption: the transformation exists and it is **linear**. This gives a change of coordinates that can be computed using matrix decomposition.

Mathematically, we start from the original data matrix $\mathbf{X} \in \mathbb{R}^{m\times n}$, and look for a transformation matrix $\mathbf{P} \in \mathbb{R}^{m\times m}$ that produces a new representation $\mathbf{Y} \in \mathbb{R}^{m\times n}$.

Let $\mathbf{x}_i$ denote the columns of $\mathbf{X}$ and let $\mathbf{p}_i$ denote the rows of $\mathbf{P}$, and $\mathbf{y}_i$ the columns of $\mathbf{Y}$. Then, the transformed dataset can be expressed as:

$$\mathbf{Y} = \mathbf{P}\mathbf{X}.$$

Explicitly, we can write

$$\mathbf{P}\mathbf{X} = \begin{bmatrix}\mathbf{p}_1\\ \vdots\\ \mathbf{p}_m\end{bmatrix}\begin{bmatrix}\mathbf{x}_1 & \dots & \mathbf{x}_n\end{bmatrix} = \begin{bmatrix}\mathbf{p}_1\mathbf{x}_1 & \dots & \mathbf{p}_1\mathbf{x}_n\\ \vdots & \ddots & \vdots\\ \mathbf{p}_m\mathbf{x}_1 & \dots & \mathbf{p}_m\mathbf{x}_n\end{bmatrix} \triangleq \begin{bmatrix}\mathbf{y}_1 & \dots & \mathbf{y}_n\end{bmatrix}.$$

Each transformed sample $\mathbf{y}_i$ therefore contains the coordinates of $\mathbf{x}_i$ in the new basis $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_m\}$. Once these directions are determined according to the PCA criterion, they are referred to as the **principal directions** of the dataset $\mathbf{X}$.

In summary, PCA aims to automatically determine a matrix $\mathbf{P}$ that transforms the original data $\mathbf{X}$ into a new representation$\mathbf{Y}$. However, we still need to define what makes the new representation better, which will lead us to consider variance and redundancy.

1.2.1 Noise and Variance

Noise is typically something that disrupts or confounds useful data from the rest. The measurement noise in any data set must be low, otherwise no meaningful info on the data can be extracted, no matter the used techniques.

There is no absolute scale for noise. We rather compare its power against that of the useful component by defining the **Signal-to-Noise Ratio (SNR)** expressed as

$$\text{SNR} = \frac{\sigma^2_{\text{signal}}}{\sigma^2_{\text{noise}}}.$$

A high SNR means that the signal dominates the noise, indicating a high-precision measure. In the spring example, the useful motion mainly varies along one direction, while noise produces a spread around the ideal trajectory (see Figure 1.2).

This motivates the search for directions of maximum variance, which we expect to align with the highest signal. Based on this intuition, we then assume that the dynamics of interest lie along directions with the high variance. This assumption, which is very strong but is needed to automate the algorithm, suggests that the direction that maximizes the variance corresponds to the best-fit for the data cloud. Eventually, we will generalize it for any dimension.

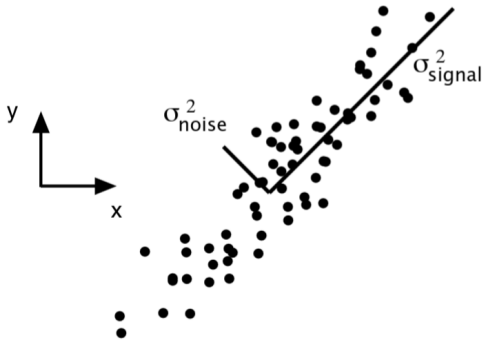


Figure 1.2: Simulated data of (x, y) for camera A.

1.2.2 Redundancy

Redundancy is another confounding factor that occurs when variables contain overlapping information. If two measurements are strongly correlated, knowing one helps predict the other. PCA reorganizes shared variability into uncorrelated components, which can make the lower-dimensional representation possible. Figure 1.3 shows three possible cases of redundancy.



(a) Low redundancy. (b) Medium redundancy. (c) High redundancy.

Figure 1.3: Different levels of redundancy in the data.

1.3 Covariance and Redundancy

To describe linear relationships between measurements, we use **covariance**. Consider two centered variables A and B, each containing n observations:

$$A = \{ a_1, a_2, \ldots, a_n \}, \qquad B = \{ b_1, b_2, \ldots, b_n \}.$$

Their variances and covariance are

$$\sigma_A^2 = \frac{1}{n}\sum_{i=1}^{n} a_i^2 \qquad \sigma_B^2 = \frac{1}{n}\sum_{i=1}^{n} b_i^2, \qquad \mathrm{Cov}(A,B) = \sigma_{A,B}^2 = \frac{1}{n}\sum_{i=1}^{n} a_i b_i.$$

Positive covariance means that the variables tend to vary in the same direction. Negative covariance means that they tend to vary in opposite directions. Zero covariance means that they are uncorrelated, but not necessarily independent. Thus, $\sigma_{A,B}^2 = 0$ if and only if A and B are uncorrelated, while if $A = B$, then $\sigma_{A,B}^2 = \sigma_A^2$.

For row vectors $\mathbf{a}$ and $\mathbf{b}$ containing the observations, the covariance is given by their inner product:

$$\mathrm{Cov}(\mathbf{a},\mathbf{b}) = \sigma_{\mathbf{a},\mathbf{b}}^2 = \frac{1}{n}\mathbf{a}\mathbf{b}^T.$$

For m variables, this can be easily generalized by collecting these quantities in the covariance matrix. Given a zero-centered data matrix[1] $\mathbf{X} \in \mathbb{R}^{m\times n}$

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \ldots & x_{1n} \\ x_{21} & x_{22} & \ldots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \ldots & x_{mn} \end{bmatrix},$$

the covariance matrix is

$$\mathbf{C}_\mathbf{X} = \frac{1}{n}\sum_{i=1}^{n}(\mathbf{x}_i - \overline{\mathbf{x}})(\mathbf{x}_i - \overline{\mathbf{x}})^T = \frac{1}{n}\mathbf{X}\mathbf{X}^T. \tag{1.2}$$

This matrix has some nice properties:

- It is an $m \times m$ symmetric matrix.
- Its diagonal entries are the variances of a measurement types.
- Its off-diagonal entries are the covariance between different measurement types.

1.3.1 What Should the New Representation Look Like?

We seek an orthonormal change of basis $\mathbf{P}$ that makes the transformed covariance matrix diagonal. The new coordinates are then uncorrelated. We also order them by decreasing variance, so that the first components capture as much variance as possible. Since we are assuming $\mathbf{Y} = \mathbf{P}\mathbf{X}$, its covariance is

$$\begin{aligned} \mathbf{C}_\mathbf{Y} &= \frac{1}{n}\mathbf{Y}\mathbf{Y}^T = \frac{1}{n}(\mathbf{P}\mathbf{X})(\mathbf{P}\mathbf{X})^T \\ &= \frac{1}{n}\mathbf{P}\mathbf{X}\mathbf{X}^T\mathbf{P}^T = \mathbf{P}\left(\frac{1}{n}\mathbf{X}\mathbf{X}^T\right)\mathbf{P}^T = \mathbf{P}\mathbf{C}_\mathbf{X}\mathbf{P}^T. \end{aligned} \tag{1.3}$$

[1] Remember that the row index i is a particular data type, and the column index j is the sample number (time).

An orthonormal change of basis preserves total variance. The goal is therefore not to increase every variance, but to concentrate the largest possible share in the first few coordinates.

1.4 Maximum Variance Formulation

We now go through one of the formulations of PCA, namely the **maximum projected variance formulation** (we will see other formulations).

We said that PCA seeks a lower-dimensional subspace such that the orthogonal projection of the data points (red dots in Figure 1.4) onto it maximizes the variance of the projected points (green dots). This is known as the **principal subspace**. In this example, it is represented by the magenta line and is spanned by the unit vector $\mathbf{u}_1$.

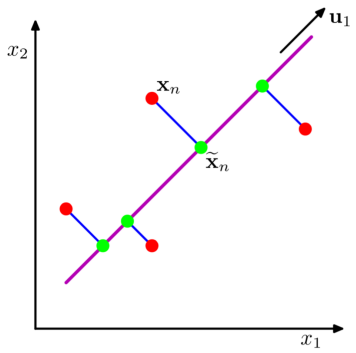


Figure 1.4

First, we look for a unit direction $\mathbf{u}_1$ such that the orthogonal projections of the data onto this direction have maximum variance. This gives the first principal direction.

For a unit vector $\mathbf{u}_1$, the scalar projection of $\mathbf{x}_i$ is $\mathbf{u}_1^T\mathbf{x}_i$. Its vector projection onto the same direction is $(\mathbf{u}_1^T\mathbf{x}_i)\mathbf{u}_1$. The projected mean is $\mathbf{u}_1^T\overline{\mathbf{x}}$, while the projected variance is

$$\sigma^2(\mathbf{u}_1) = \frac{1}{n}\sum_{i=1}^{n}\left[\mathbf{u}_1^T(\mathbf{x}_i - \overline{\mathbf{x}})\right]^2 = \mathbf{u}_1^T\mathbf{C}_\mathbf{X}\mathbf{u}_1,$$

where $\mathbf{C}_\mathbf{X}$ is the covariance matrix defined in Equation 1.2.

Proof. The variance of data projected onto direction $\mathbf{u}_1$ is:

$$\begin{aligned}\sigma^2(\mathbf{u}_1) &= \frac{1}{n}\sum_{i=1}^{n}(\mathbf{u}_1^T\mathbf{x}_i - \mathbf{u}_1^T\overline{\mathbf{x}})^2 = \frac{1}{n}\sum_{i=1}^{n}(\mathbf{u}_1^T(\mathbf{x}_i - \overline{\mathbf{x}}))(\mathbf{u}_1^T(\mathbf{x}_i - \overline{\mathbf{x}}))^T = \\ &= \frac{1}{n}\sum_{i=1}^{n}\mathbf{u}_1^T(\mathbf{x}_i - \overline{\mathbf{x}})(\mathbf{x}_i - \overline{\mathbf{x}})^T\mathbf{u}_1 = \mathbf{u}_1^T\left[\frac{1}{n}\sum_{i=1}^{n}(\mathbf{x}_i - \overline{\mathbf{x}})(\mathbf{x}_i - \overline{\mathbf{x}})^T\right]\mathbf{u}_1 = \\ &= \mathbf{u}_1^T\mathbf{C}_\mathbf{X}\mathbf{u}_1.\end{aligned}$$

□

We now maximize this quantity subject to the unit-length (normalization) condition. This is an optimization problem that can be re-expressed as:

$$\max_{\mathbf{u}_1}\mathbf{u}_1^T\mathbf{C}_\mathbf{X}\mathbf{u}_1 \qquad \text{subject to:} \qquad \mathbf{u}_1^T\mathbf{u}_1 = 1.$$

Therefore, the Lagrange multiplier gives $L(\mathbf{u}_1)$:

$$L(\mathbf{u}_1, \lambda_1) = \mathbf{u}_1^T \mathbf{C}_\mathbf{X} \mathbf{u}_1 + \lambda_1 (1 - \mathbf{u}_1^T \mathbf{u}_1).$$

The stationary conditions of the Lagrangian are[2]

$$\frac{\partial L(\mathbf{u}_1)}{\partial \mathbf{u}_1} = 2\mathbf{C}_\mathbf{X} \mathbf{u}_1 - 2\lambda_1 \mathbf{u}_1 = 0, \tag{1.4a}$$

$$\frac{\partial L(\mathbf{u}_1)}{\partial \lambda_1} = 1 - \mathbf{u}_1^T \mathbf{u}_1 = 0 \quad \Rightarrow \quad \mathbf{u}_1^T \mathbf{u}_1 = 1. \tag{1.4b}$$

Rearranging Equation 1.4a gives

$$\mathbf{C}_\mathbf{X} \mathbf{u}_1 = \lambda_1 \mathbf{u}_1 \tag{1.5}$$

which is an eigenvalue equation. This means that the stationary directions are eigenvectors of the covariance matrix. By left-multiplying Equation 1.5 by $\mathbf{u}_1^T$, we get

$$\mathbf{u}_1^T \mathbf{C}_\mathbf{X} \mathbf{u}_1 = \lambda_1.$$

Thus, the variance along a unit eigenvector equals its eigenvalue. The maximum is obtained by selecting an eigenvector associated with the largest eigenvalue. This procedure can be iterated for the other dimensions and the next principal direction maximizes projected variance while being orthogonal to the first.

Observation. Notice that covariance values reflect the noise and redundancy in the measurements that we discussed previously. In the diagonal terms, high values mean interesting structure of data that we would like to capture and to emphasize. Meanwhile, in the off-diagonal terms, large magnitudes imply high redundancy that we want to minimize as much as possible in order to *clean* the dataset.

These insights are important because they reflect the structure we want the covariance matrix to have. More precisely, $\mathbf{C}_\mathbf{X}$ is determined by the original data, but we can choose a transformation $\mathbf{P}$ to obtain a new covariance matrix $\mathbf{C}_\mathbf{Y}$ with the desired properties. We therefore seek a transformation that makes the off-diagonal entries zero and concentrates as much variance as possible in the first few diagonal entries.

Objective revisited. With the problem now better defined and structured, we can revisit and clearly state our objectives. We want to

1. minimize the redundancy, measured by the covariances,
2. maximize the signal power (SNR), measured by the variance.

Returning to the transformation we seek, we have:

$$\mathbf{PX} = \mathbf{Y} \quad \Rightarrow \quad \mathbf{P}\mathbf{C}_\mathbf{X}\mathbf{P}^T = \mathbf{C}_\mathbf{Y}$$

The optimized covariance matrix $\mathbf{C}_\mathbf{Y}$ should have all its off-diagonal elements close to zero because $\mathbf{Y}$ should be uncorrelated (1), and each subsequent dimension in $\mathbf{Y}$ should be rank-ordered according to variance, from largest to smallest (2).

[2]The gradient of a quadratic form is derived in Appendix A.

1.5 PCA through Eigendecomposition

Instead of finding the principal directions one at a time, we can obtain them from the eigendecomposition of the covariance matrix. To do this, we must diagonalize the covariance $\mathbf{C_X}$ in the transformed domain.

1.5.1 Eigendecomposition of a Symmetric Matrix

To find the transformation $\mathbf{P}$ that makes $\mathbf{C_Y}$ we use the following theorem without giving its proof.

Theorem 1.5.1 (Eigenvector Decomposition)**.** *Every real symmetric matrix* $\mathbf{A} \in \mathbb{R}^{n\times n}$ *admits an orthonormal basis of eigenvectors. If* $\mathbf{E}$ *contains these eigenvectors as columns, then*

$$\mathbf{E}^T\mathbf{A}\mathbf{E} = \mathbf{D}, \qquad \mathbf{A} = \mathbf{E}\mathbf{D}\mathbf{E}^T,$$

where

$$\mathbf{D} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n) = \begin{bmatrix} \lambda_1 & 0 & \ldots & 0 \\ 0 & \lambda_2 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & \lambda_n \end{bmatrix}$$

contains the corresponding eigenvalues and $\mathbf{E}^T\mathbf{E} = \mathbf{I}$.

This theorem states that $\mathbf{A}$ can be written as the product of an orthogonal matrix $\mathbf{E}$ and a diagonal matrix $\mathbf{D}$ containing its eigenvalues by construction.

Since $\mathbf{C_X}$ is symmetric and positive semidefinite, we can apply this decomposition and order the eigenvalues as

$$\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_m \geq 0.$$

Let $\mathbf{u}_j$ be its unit eigenvectors, collected as columns of $\mathbf{E}$. We choose

$$\mathbf{P} = \mathbf{E}^T = \begin{bmatrix} \mathbf{u}_1^T \\ \mathbf{u}_2^T \\ \vdots \\ \mathbf{u}_m^T \end{bmatrix}.$$

Thus, the row vector $\mathbf{p}_j$ used earlier is $\mathbf{u}_j^T$. The matrix $\mathbf{P}$ is orthogonal, so $\mathbf{P}^{-1} = \mathbf{P}^T$.

With this choice of $\mathbf{P}$, we can diagonalize $\mathbf{C_Y}$ following Equation 1.3:

$$\begin{aligned} \mathbf{C_Y} = \mathbf{P}\mathbf{C_X}\mathbf{P}^T = \mathbf{P}\left(\frac{1}{n}\mathbf{X}\mathbf{X}^T\right)\mathbf{P}^T = \mathbf{P}(\mathbf{E}\mathbf{D}\mathbf{E}^T)\mathbf{P}^T \overset{*}{=} \\ = \mathbf{P}(\mathbf{P}^T\mathbf{D}\mathbf{P})\mathbf{P}^T = \mathbf{P}\mathbf{P}^T\mathbf{D}\mathbf{P}\mathbf{P}^T = \mathbf{D}, \end{aligned}$$

where in $*$ we applied Theorem 1.5.1. The transformed coordinates are then uncorrelated, and their variances are the eigenvalues of $\mathbf{C_X}$. The eigenvectors are the principal directions, while the projected coordinates are the component scores.

1.5.2 Practical Steps

Therefore, the practical steps for PCA are:

1. Organize the data into an $m \times n$ matrix $\mathbf{X}$, where m is the number of measurement types, and n number of samples.
2. Compute the mean vector: $\overline{\mathbf{x}}$ and subtract it from each sample: $\mathbf{x}_i \leftarrow \mathbf{x}_i - \overline{\mathbf{x}}$.
3. Compute the covariance matrix: $\mathbf{C}_{\mathbf{X}} = \frac{1}{n}\mathbf{X}\mathbf{X}^T$.
4. Compute its eigenvectors and eigenvalues.
5. Sort them by decreasing eigenvalue and form $\mathbf{P}$ from the transposed eigenvectors.
6. Apply change of basis and transform the centered data using $\mathbf{PX} = \mathbf{Y}$.

End. $\mathbf{Y}$ is the transformed data matrix and for dimensionality reduction, retain only the first $K < m$ rows of $\mathbf{Y}$ and the corresponding directions.

The eigendecomposition of a dense $m \times m$ covariance matrix has a computational cost of order $\mathcal{O}(m^3)$. This is only the decomposition step, not the cost of the entire procedure. However, in general the number of measurement types is much smaller than the number of samples, i.e., $m \ll n$.

1.6 Dimensionality Reduction and Reconstruction

PCA can be very useful for data compression. Consider a dataset with m original dimensions. Keeping all principal components only changes the representation of the data, without losing information. To reduce dimensionality, we retain the K components associated with the largest eigenvalues, where $K < m$. The following derivation can be found in [2].

1.6.1 Exact Reconstruction

Consider n samples $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$ with $\mathbf{x}_i \in \mathbb{R}^m$. Once matrix $\mathbf{P}$ is found for each input vector $\mathbf{x}_i$, the corresponding transformed output vector is:

$$\mathbf{y}_i = \mathbf{P}(\mathbf{x}_i - \overline{\mathbf{x}}) = \mathbf{P}\mathbf{x}'_i \in \mathbb{R}^m, \qquad y_{ji} = \mathbf{p}_j \mathbf{x}'_i = \mathbf{u}_j^T \mathbf{x}'_i. \tag{1.6}$$

where $\mathbf{x}'_i$ is the centered sample. The index j identifies the component, and i identifies the sample. Since the principal directions form an orthonormal basis, each sample can be expressed exactly using the new basis, that is,

$$\mathbf{x}'_i = \sum_{k=1}^{m} \alpha_{ki} \mathbf{u}_k. \tag{1.7}$$

The coefficients α_{ki} are scalars that differ for each vector. This operation corresponds to a rotation of the original coordinate system onto a new system defined by the $\mathbf{u}_k$. To find the coefficients, we left-multiply the expansion of $\mathbf{x}'_i$ in Equation 1.7 by $\mathbf{u}_j^T$

$$\mathbf{u}_j^T \mathbf{x}'_i = \sum_{k=1}^{m} \alpha_{ki} \mathbf{u}_j^T \mathbf{u}_k.$$

Since the basis is orthonormal,

$$\mathbf{u}_j^T \mathbf{u}_k = \delta_{jk}, \tag{1.8}$$

we obtain

$$\mathbf{u}_j^T \mathbf{x}_i' = \alpha_{ji} \quad \Rightarrow \quad \alpha_{ji} = y_{ji}.$$

Therefore, the original data can be exactly retrieved from $\mathbf{y}$ as

$$\mathbf{x}_i' = \sum_{k=1}^{m} y_{ki} \mathbf{u}_k,$$

and adding back the mean vector leads us to

$$\mathbf{x}_i = \overline{\mathbf{x}} + \mathbf{x}_i' = \overline{\mathbf{x}} + \sum_{k=1}^{m} y_{ki} \mathbf{u}_k. \tag{1.9}$$

Equation 1.9 is an exact equality and it means that we can re-express the original data point in terms of the average information of the dataset plus a term that depends on the new basis of the transformed domain. This means that the representation in the two domains are exactly equivalent and all components preserve the original information.

1.6.2 Approximation and Reconstruction Error

The previous result allows us to approximate the original data by retaining the first K components in the transformed vector, with $K < m$. This gives

$$\tilde{\mathbf{x}}_i = \overline{\mathbf{x}} + \tilde{\mathbf{x}}_i' = \overline{\mathbf{x}} + \sum_{k=1}^{K} y_{ki} \mathbf{u}_k,$$

and the gap between the original and the approximation is

$$\mathbf{x}_i - \tilde{\mathbf{x}}_i = \sum_{k=K+1}^{m} y_{ki} \mathbf{u}_k. \tag{1.10}$$

A metric to quantify the distortion between original and approximated data is the **distortion measure** J, which basically uses the mean squared reconstruction error:

$$J = \frac{1}{n} \sum_{i=1}^{n} \left\| \mathbf{x}_i - \tilde{\mathbf{x}}_i \right\|^2.$$

For a fixed K, retaining the largest eigenvalues leaves the smallest possible sum of discarded eigenvalues. PCA therefore minimizes mean squared reconstruction error among K-dimensional linear subspaces of the centered data. Thus, it holds

$$J = \sum_{k=K+1}^{m} \lambda_k,$$

which is the sum of eigenvalues of discarded dimensions.

Proof.

$$J = \frac{1}{n}\sum_{i=1}^{n}\|\mathbf{x}_i - \tilde{\mathbf{x}}_i\|^2 = \frac{1}{n}\sum_{i=1}^{n}\left\|\sum_{k=K+1}^{m} y_{ki}\mathbf{u}_k\right\|^2 = \frac{1}{n}\sum_{i=1}^{n}\left(\sum_{k=K+1}^{m} y_{ki}\mathbf{u}_k\right)^T\left(\sum_{r=K+1}^{m} y_{ri}\mathbf{u}_r\right) =$$

$$= \frac{1}{n}\sum_{i=1}^{n}\left(\sum_{k=K+1}^{m} y_{ki}\mathbf{u}_k^T\right)\left(\sum_{r=K+1}^{m} y_{ri}\mathbf{u}_r\right) = \frac{1}{n}\sum_{i=1}^{n}\left(\sum_{k,r=K+1}^{m} y_{ki}y_{ri}\mathbf{u}_k^T\mathbf{u}_r\right) =$$

$$\overset{(1.8)}{=} \frac{1}{n}\sum_{i=1}^{n}\sum_{k=K+1}^{m} y_{ki}y_{ki} \overset{(1.6)}{=} \frac{1}{n}\sum_{i=1}^{n}\sum_{k=K+1}^{m}\mathbf{u}_k^T\mathbf{x}_i'(\mathbf{x}_i')^T\mathbf{u}_k = \sum_{k=K+1}^{m}\mathbf{u}_k^T\left(\frac{1}{n}\sum_{i=1}^{n}\mathbf{x}_i'(\mathbf{x}_i')^T\right)\mathbf{u}_k =$$

$$\overset{(1.2)}{=} \sum_{k=K+1}^{m}\lambda_k.$$

□

1.7 Summary of PCA Assumptions

- **Linearity** frames the problem as a *change of basis*. Extensions to nonlinear representations have also been explored, for example through autoencoders.
- **Large variance reveals important structure**: we assume that the useful signal dominates the noise along the directions of greatest variability. Under this assumption, components with larger variances capture meaningful structure, while those with smaller variances may mainly represent noise.
- **The principal directions are orthogonal**, forming an orthonormal basis when normalized. Together with the variance criterion, this allows PCA to be solved using linear algebra decomposition techniques.

1.8 Applications

1.8.1 Handwritten digits

Handwritten digit recognition is a classic problem in machine learning. The MNIST dataset contains 70,000 images of handwritten digits. The digits were size-normalized and centered in 28×28 pixel grayscale images. Each image can therefore be represented as a vector in $\mathbb{R}^{784}$. Examples of these digits are shown in Figure 1.5.

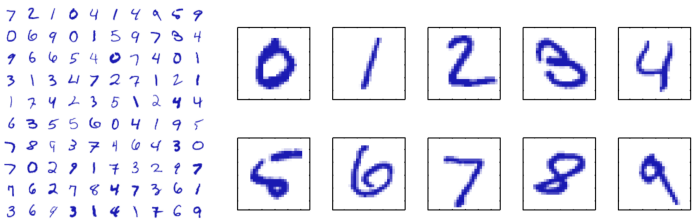
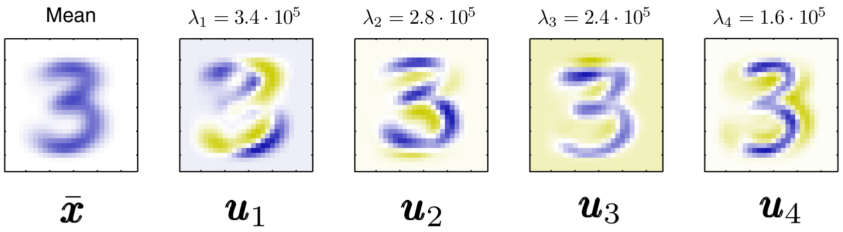


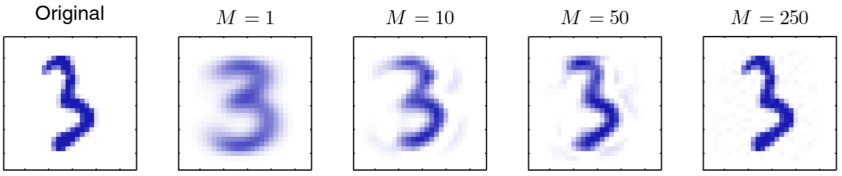
Figure 1.5: MNIST dataset.

Let us apply PCA to the samples representing the digit 3. The mean image summarizes their average appearance (leftmost panel in Figure 1.6a), while the principal directions (eigenvectors) describe the variations around that mean.

Figure 1.6b shows the reconstruction of a sample using M principal components. Keeping only a few components gives an imperfect approximation, but the digit is still recognizable. As the number of retained components increases, the reconstruction error deacreases and more details are recovered.



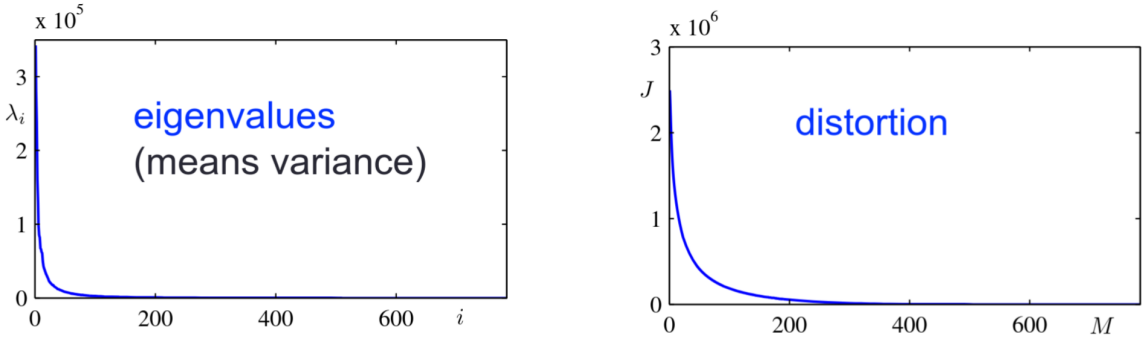
(a) Mean and first four strongest eigenvalues and corresponding eigenvectors of number 3.



(b) Example of PCA reconstruction obtained by retaining M principal components

Figure 1.6

Figure 1.7 shows both the distribution of the eigenvalues ordered from largest to smallest and the distortion as a function of the number of retained components. The left panel shows that most of the variance is concentrated in the first few principal components. The right panel shows that the distortion decreases as more components are retained, reaching zero when $M = 784$, which is the full dimensionality of each image.



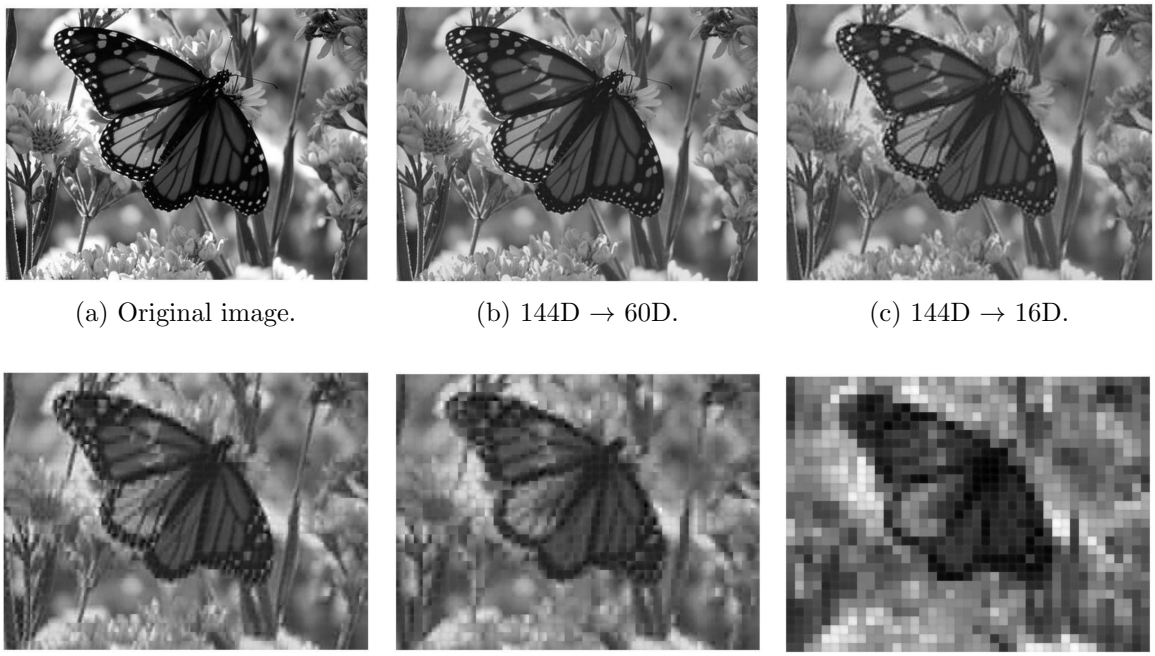
(a) Eigenvalues vs order.

(b) Distortion vs data size.

Figure 1.7

1.8.2 Image Compression

As a practical example for image compression, Figure 1.8 shows the image (372×492 pixels) of a butterfly. To compress the information of the image, we can divide it into 1271 patches of 12×12 pixels. Each patch is represented by a $12 \times 12 = 144$-dimensional vector. PCA is then applied to all patches, providing a shared basis for their representation. Each patch is reconstructed using only K components, and the reconstructed patches are assembled into an image. In Figure 1.8b-1.8f, PCA retains 60, 16, 6, 3, and 1 component per patch. As K decreases, fewer coefficients are retained and more detail is lost.



(a) Original image. (b) 144D $\rightarrow$ 60D. (c) 144D $\rightarrow$ 16D.

(d) 144D $\rightarrow$ 6D. (e) 144D $\rightarrow$ 3D. (f) 144D $\rightarrow$ 1D.

Figure 1.8: 372×492 pixels image divided into 1271 patches. The original patch size is 144.

This operation is called *lossy compression*: fewer coefficients are stored at the cost of reconstruction error in a controlled manner, quantifying how much information we lose. To reconstruct the patches, we need the mean patch and the retained basis vectors.



(a) 60D basis for the butterfly image. (b) DCT basis for jpg.

Figure 1.9

In Figure 1.9, PCA basis vectors of the butterfly image are compared with the basis functions of the **discrete cosine transform (DCT)**, used in standard JPG compression. The key difference is that the DCT basis is fixed, while the PCA basis is computed from the data and then it can be *customized* to a particular dataset.

For instance, lossy compression may be useful in medical field, where one would like to have images that have sufficiently resolution to identify problems but can be easily transmitted without losing too much information.

1.9 Where PCA Fails

In general, PCA cannot represent non-linear structure using a low-dimensional linear subspace. This is where it typically fails.

An example of problematic datasets is shown in Figure 1.10. On the left panel, a point moves on a circle – in this example, a Ferris wheel. Although the data lie in a higher-dimensional space, the dynamics can be described by one angular variable, the phase of the wheel θ. However, a single linear projection cannot preserve its complete position on the circle. The problem is the mismatch between the curved structure and the linear representation.

The right panel shows another example where PCA fails. The data are concentrated around two intersecting, non-orthogobanl directions. PCA selectes orthogonal principal directions, shown by the red arrows. The first follows the dominant spread of the data, while the second must be perpendicular to it, and therefore does not align with the other branch.

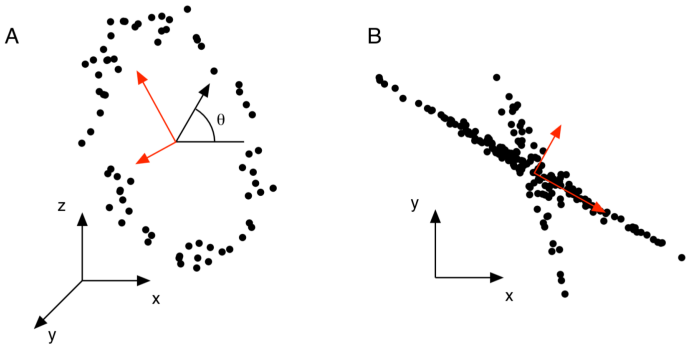


Figure 1.10

1.10 Final Remarks

PCA has widespread applications because it can reveal simple underlying structures in complex datasets using matrix decomposition techniques from linear algebra. Its main benefit is that it quantifies how much variance each principal component captures, providing a way to compare their relative importance in representing the data.

The idea behind dimensionality reduction with PCA is that a small number of principal components may capture most of the variability of the dataset. If this variability reflects the structure of interest, retaining these components provides a useful approximation with

fewer variables. As we have shown, for a fixed number of retained components, PCA minimizes the mean squared reconstruction error among linear subspaces of that dimensionality. The discarded eigenvalues quantify the resulting distortion, making the trade-off between dimensionality and reconstruction error explicit.

PCA is an unsupervised technique because it determines the principal directions without using labels. It does not require us to specify a probability distribution for the data. However, we still need to choose how many components to retain and consider the scale of the original measurements, since this affects their contribution to the variance.

PCA also decorrelates the data: the covariance matrix of the transformed dataset $\mathbf{Y}$ is diagonal by construction. This can be useful as a preprocessing step before classification or other tasks. If we additionally divide each component with nonzero variance by the square root of its eigenvalue, the resulting components have unit variance. This additional normalization, together with decorrelation, is known as **whitening**.

The usefulness of PCA therefore depends on whether a linear representation and the variance criterion are appropriate for the problem. When they are, PCA offers a simple way to reduce dimensionality, control reconstruction error, and explore the structure of the data.

Further extensions are discussed in Chapter 12 of [2]. Other applications include face recognition using eigenfaces [3] and motion tracking in mobile systems [4].

1.11 Connection with SVD

PCA is closely related to **singular value decomposition (SVD)**. Let $\mathbf{X} \in \mathbb{R}^{m \times n}$ be the centered data matrix, with $r = \operatorname{rank}(\mathbf{X}) > 0$. Define

$$\mathbf{Z} \triangleq \frac{1}{\sqrt{n}}\mathbf{X}.$$

Then,

$$\mathbf{Z}\mathbf{Z}^T = \frac{1}{n}\mathbf{X}\mathbf{X}^T = \mathbf{C}_\mathbf{X}.$$

We can apply Theorem A.5.1, given in Appendix A, to $\mathbf{Z}$. In this case, $\mathbf{Z}\mathbf{Z}^T = \mathbf{C}_\mathbf{X}$ corresponds to the matrix $\mathbf{A}\mathbf{A}^T$ in the theorem. Writing

$$\mathbf{Z} = \mathbf{U}\boldsymbol{\Sigma}_{\boldsymbol{Z}}{}^{1/2}\mathbf{V}^T,$$

the columns of $\mathbf{U}$ are orthonormal eigenvectors associated with the r positive eigenvalues of $\mathbf{C}_\mathbf{X}$, contained in $\boldsymbol{\Sigma}_{\boldsymbol{Z}}$. These columns are therefore the principal directions along which the data have non-zero variance.

The complete matrix $[\mathbf{U} \mid \mathbf{U}_*]$ corresponds to $\mathbf{E}$ in Theorem 1.5.1. The columns of $\mathbf{U}_*$ are the remaining eigenvectors, associated with zero eigenvalues. From the proof of Theorem A.5.1, we know that

$$\mathbf{Z}^T\mathbf{U}_* = \mathbf{0} \quad \Rightarrow \quad \mathbf{U}_*^T\mathbf{X} = \mathbf{0}.$$

Thus, the centered data have zero coordinates along these directions, which can be discarded without losing information.

In practice, we can apply SVD directly to $\mathbf{X}$. We first arrange the data into a matrix with m measurement types and n samples, then subtract the sample mean vector from each sample. The decomposition of the centered matrix is

$$\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}_{\boldsymbol{X}}{}^{1/2}\mathbf{V}^T.$$

Since $\mathbf{X} = \sqrt{n}\mathbf{Z}$, the matrices $\mathbf{U}$ and $\mathbf{V}$ can be chosen as before, while

$$\boldsymbol{\Sigma_X} = n\boldsymbol{\Sigma_Z}.$$

Therefore, $\boldsymbol{\Sigma_X}$ contains the positive eigenvalues of $\mathbf{X}\mathbf{X}^T$, and dividing them by n gives the corresponding eigenvalues of $\mathbf{C_X}$. We order them from largest to smallest, arranging the columns of $\mathbf{U}$ and $\mathbf{V}$ accordingly.

Multiplying the decomposition by $\mathbf{U}^T$ gives

$$\mathbf{Y} = \mathbf{U}^T\mathbf{X} = \mathbf{U}^T\mathbf{U}\boldsymbol{\Sigma_X}^{1/2}\mathbf{V}^T = \boldsymbol{\Sigma_X}^{1/2}\mathbf{V}^T.$$

The rows of $\mathbf{U}^T$ are the transposed principal directions. Thus, $\mathbf{P}_r = \mathbf{U}^T$ is the PCA transformation that retains all r components with non-zero variance:

$$\mathbf{Y} = \mathbf{P}_r\mathbf{X}, \qquad \mathbf{P}_r \in \mathbb{R}^{r\times m}, \qquad \mathbf{Y} \in \mathbb{R}^{r\times n}.$$

Compared with the full $m \times m$ transformation $\mathbf{P}$, $\mathbf{P}_r$ simply omits the directions with zero variance.

These r components are sufficient to represent the centered dataset without information loss, since

$$\mathbf{X} = \mathbf{U}\mathbf{Y}.$$

The remaining $m - r$ directions describe linear dependencies among the original measurements and contain no additional variability. In the PCA space, r uncorrelated components therefore suffice to represent the data. Retaining fewer than r components introduces a reconstruction error.

Appendix A

Useful Algebra Results

A.1 Quadratic Forms

A.1.1 Definition and Symmetric Representation

Definition A.1.1. Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be a matrix and $\mathbf{x} \in \mathbb{R}^{n\times 1}$ be a vector. A **quadratic form** is defined as

$$\alpha = \mathbf{x}^T\mathbf{A}\mathbf{x}.$$

In a quadratic form, we may assume without loss of generality that $\mathbf{A}$ is symmetric, since we can always replace it with

$$\frac{(\mathbf{A}^T + \mathbf{A})}{2}$$

Proof.

$$\mathbf{x}^T\frac{(\mathbf{A}^T + \mathbf{A})}{2}\mathbf{x} = \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} + \frac{1}{2}\mathbf{x}^T\mathbf{A}^T\mathbf{x} = \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} + \frac{1}{2}(\mathbf{x}^T\mathbf{A}\mathbf{x})^T = \\ = \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} + \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} = \mathbf{x}^T\mathbf{A}\mathbf{x}.$$

The third equality holds because $\mathbf{x}^T\mathbf{A}\mathbf{x}$ is a scalar. □

A.1.2 Definiteness

Definition A.1.2. Let $\mathbf{A}$ be a symmetric matrix, we say that $\mathbf{A}$ is

$$\begin{aligned}
\textbf{positive definite} &\text{ if } \mathbf{x}^T\mathbf{A}\mathbf{x} > 0 \quad \forall \mathbf{x} \neq 0 \\
\textbf{positive semi-definite} &\text{ if } \mathbf{x}^T\mathbf{A}\mathbf{x} \geq 0 \quad \forall \mathbf{x} \\
\textbf{negative definite} &\text{ if } \mathbf{x}^T\mathbf{A}\mathbf{x} < 0 \quad \forall \mathbf{x} \neq 0 \\
\textbf{negative semi-definite} &\text{ if } \mathbf{x}^T\mathbf{A}\mathbf{x} \leq 0 \quad \forall \mathbf{x} \\
\textbf{indefinite} &\text{ if } \mathbf{x}^T\mathbf{A}\mathbf{x} \gtrless 0 \quad \text{for some } \mathbf{x}
\end{aligned}$$

A.1.3 Gradient of a Quadratic Form

Theorem A.1.3 (Gradient of a Quadratic Form)**.** *Let α be the scalar quadratic form*

$$\alpha \triangleq \mathbf{x}^T \mathbf{A} \mathbf{x},$$

where $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{A} = [a_{ij}] \in \mathbb{R}^{n \times n}$ and does not depend on $\mathbf{x}$.

We define the gradient of α as the row vector:

$$\nabla \alpha(\mathbf{x}) \triangleq \left(\frac{\partial \alpha}{\partial x_1}, \frac{\partial \alpha}{\partial x_2}, \ldots, \frac{\partial \alpha}{\partial x_n} \right).$$

Then,

$$\nabla \alpha(\mathbf{x}) = \mathbf{x}^T (\mathbf{A} + \mathbf{A}^T).$$

Proof. By definition,

$$\alpha = \sum_{ij} a_{ij} x_i x_j.$$

Differentiating with respect to the k-th element of $\mathbf{x}$, we get:

$$\frac{\partial \alpha}{\partial x_k} = \sum_j a_{kj} x_j + \sum_i a_{ik} x_i, \qquad \forall k = 1, 2, \ldots, n.$$

This can be compactly written as:

$$\nabla \alpha(\mathbf{x}) = \mathbf{x}^T (\mathbf{A} + \mathbf{A}^T).$$

□

If $\mathbf{A}$ is symmetric ($\mathbf{A} = \mathbf{A}^T$), we have:

$$\nabla \alpha(\mathbf{x}) = \mathbf{x}^T (\mathbf{A} + \mathbf{A}^T) = \mathbf{x}^T (2\mathbf{A}) = 2\mathbf{x}^T \mathbf{A}.$$

In column form, we get:

$$(\nabla \alpha(\mathbf{x}))^T = (2\mathbf{x}^T \mathbf{A})^T = 2\mathbf{A}^T \mathbf{x} = 2\mathbf{A}\mathbf{x}.$$

A.1.4 Gradient of the Squared Norm

Consider the squared Euclidean norm

$$\alpha = \mathbf{x}^T \mathbf{x} = \sum_i x_i^2.$$

Differentiating with respect to x_j gives

$$\frac{\partial \alpha}{\partial x_j} = 2x_j.$$

In column vector form,

$$(\nabla \alpha(\mathbf{x}))^T = 2\mathbf{x}.$$

This is the previous case with $\mathbf{A} = \mathbf{I}$.

A.2 Eigenvalues and Eigenvectors

A.2.1 Linear Independence of Eigenvectors

Theorem A.2.1. *Two eigenvectors associated with distinct eigenvalues are linearly independent.*

Proof. Let

$$\mathbf{A}\mathbf{u}_1 = \lambda_1\mathbf{u}_1, \quad \mathbf{A}\mathbf{u}_2 = \lambda_2\mathbf{u}_2,$$

with $\lambda_1 \neq \lambda_2$. Assume $\mathbf{u}_1$ and $\mathbf{u}_2$ are linearly dependent, i.e., $\mathbf{u}_2 = \beta\mathbf{u}_1$ for some $\beta \neq 0$. Then,

$$\beta\lambda_1\mathbf{u}_1 = \beta\mathbf{A}\mathbf{u}_1 = \mathbf{A}\mathbf{u}_2 = \lambda_2\mathbf{u}_2 = \beta\lambda_2\mathbf{u}_1.$$

This implies

$$\beta(\lambda_1 - \lambda_2)\mathbf{u}_1 = \mathbf{0}.$$

Since $\beta \neq 0$, and $\lambda_1 \neq \lambda_2$, this would imply

$$\mathbf{u}_1 = \mathbf{0} \quad \Rightarrow \quad \mathbf{u}_2 = \mathbf{0}$$

and, in turn, $\lambda_1 = \lambda_2$, which is a contradiction. □

A.2.2 Orthogonality for Symmetric Matrices

Theorem A.2.2. *Eigenvectors of a real symmetric matrix associated with distinct eigenvalues are orthogonal.*

Proof. Let

$$\mathbf{A}\mathbf{u}_1 = \lambda_1\mathbf{u}_1, \quad \mathbf{A}\mathbf{u}_2 = \lambda_2\mathbf{u}_2$$

with $\lambda_1 \neq \lambda_2$. Since $\mathbf{A}$ is symmetric,

$$\begin{aligned}\lambda_1\mathbf{u}_1 \cdot \mathbf{u}_2 = (\lambda_1\mathbf{u}_1)^T\mathbf{u}_2 = (\mathbf{A}\mathbf{u}_1)^T\mathbf{u}_2 = \\ = \mathbf{u}_1^T\mathbf{A}^T\mathbf{u}_2 = \mathbf{u}_1^T(\mathbf{A}\mathbf{u}_2) = \\ = \mathbf{u}_1^T(\lambda_2\mathbf{u}_2) = \lambda_2\mathbf{u}_1 \cdot \mathbf{u}_2.\end{aligned}$$

This implies that

$$(\lambda_1 - \lambda_2)\mathbf{u}_1 \cdot \mathbf{u}_2 = 0 \quad \Rightarrow \quad \mathbf{u}_1 \cdot \mathbf{u}_2 = 0$$

□

For repeated eigenvalues, an orthonormal basis can be chosen within each eigenspace. Thus, a real symmetric matrix admits an orthonormal basis of eigenvectors, as stated in Theorem 1.5.1.

A.2.3 Eigenvalues and Definiteness

Theorem A.2.3 (Symmetric Matrices)**.** *A real symmetric matrix is positive definite (semi-definite) if and only if all of its eigenvalues are positive (non-negative).*

Proof. $\Rightarrow$ Assume $\mathbf{A}$ is positive definite and let $\mathbf{x} \neq \mathbf{0}$ be an eigenvector:

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}.$$

Multiplying by $\mathbf{x}^T$ gives

$$\mathbf{x}^T\mathbf{A}\mathbf{x} = \lambda\mathbf{x}^T\mathbf{x}.$$

By assumption $\mathbf{x}^T\mathbf{A}\mathbf{x} > 0$. Since $\mathbf{x}^T\mathbf{x} > 0$, we obtain $\lambda > 0$. This holds for every eigenvalue.

$\Leftarrow$ Now assume all the eigenvalues of $\mathbf{A}$ are positive. From Theorem 1.5.1,

$$\mathbf{E}^T\mathbf{A}\mathbf{E} = \mathbf{D} = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_n \end{bmatrix},$$

where the columns of $\mathbf{E}$ form an orthonormal basis of eigenvectors of $\mathbf{A}$. Hence,

$$\mathbf{E}^T\mathbf{A}\mathbf{E} = \mathbf{D} \quad \leftrightarrow \quad \mathbf{A} = \mathbf{E}\mathbf{D}\mathbf{E}^T.$$

With the change of variable $\mathbf{u} = \mathbf{E}\mathbf{v}$

$$\mathbf{u}^T\mathbf{A}\mathbf{u} = \mathbf{v}^T\mathbf{E}^T\mathbf{E}\mathbf{D}\mathbf{E}^T\mathbf{E}\mathbf{v} = \mathbf{v}^T\mathbf{D}\mathbf{v} = \sum_{i=1}^{n} \lambda_i v_i^2.$$

Since $\mathbf{E}$ is orthogonal, $\mathbf{u} \neq 0$ implies $\mathbf{v} \neq 0$. At least one v_i is therefore non-zero, and all λ_i are positive. It follows that

$$\mathbf{u}^T\mathbf{A}\mathbf{u} > 0 \qquad \forall \mathbf{u} \neq \mathbf{0}.$$

For the positive semi-definite case, the forward direction ($\Rightarrow$) gives $\lambda \geq 0$. Conversely, if all $\lambda_i \geq 0$, the same sum is non-negative for every $\mathbf{v}$. □

A.3 Convecity of Quadratic Forms

Theorem A.3.1. *A quadratic form $f(\mathbf{x}) = \mathbf{x}^T\mathbf{Q}\mathbf{x}$, where $\mathbf{Q}$ is symmetric and positive semi-definite matrix, is convex.*

Proof. A necessary and sufficient condition for convexity is, for all points in the domain of the function

$$f(\mathbf{x}_1) - f(\mathbf{x}_2) \geq (\mathbf{x}_1 - \mathbf{x}_2)^T(\nabla f(\mathbf{x}_2))^T \qquad \forall \mathbf{x}_1, \mathbf{x}_2.$$

Here, the transpose of the gradient appears because we defined the gradient as a row vector.

Since $\mathbf{Q}$ is symmetric and positive semi-definite, there exists a matrix $\mathbf{C}$ such that the following decomposition holds

$$\mathbf{Q} = \mathbf{C}^T\mathbf{C}.$$

Indeed, from $\mathbf{Q} = \mathbf{E}\mathbf{D}\mathbf{E}^T$, we can choose $\mathbf{C} = \mathbf{D}^{1/2}\mathbf{E}^T$, since the eigenvalues in $\mathbf{D}$ are non-negative.

Now, consider the following squared norm, which is non-negative

$$\begin{aligned}\|\mathbf{C}\mathbf{x}_1 - \mathbf{C}\mathbf{x}_2\|^2 &= (\mathbf{C}\mathbf{x}_1 - \mathbf{C}\mathbf{x}_2)^T(\mathbf{C}\mathbf{x}_1 - \mathbf{C}\mathbf{x}_2) = \\ &= \mathbf{x}_1^T\mathbf{C}^T\mathbf{C}\mathbf{x}_1 - 2\mathbf{x}_1^T\mathbf{C}^T\mathbf{C}\mathbf{x}_2 + \mathbf{x}_2^T\mathbf{C}^T\mathbf{C}\mathbf{x}_2 = \\ &= \mathbf{x}_1^T\mathbf{Q}\mathbf{x}_1 - 2\mathbf{x}_1^T\mathbf{Q}\mathbf{x}_2 + \mathbf{x}_2^T\mathbf{Q}\mathbf{x}_2 \geq 0\end{aligned}$$

Adding and subtracting $\mathbf{x}_2^T\mathbf{Q}\mathbf{x}_2$, we can rewite it as

$$\begin{aligned}&\mathbf{x}_1^T\mathbf{Q}\mathbf{x}_1 - 2\mathbf{x}_1^T\mathbf{Q}\mathbf{x}_2 + 2\mathbf{x}_2^T\mathbf{Q}\mathbf{x}_2 - \mathbf{x}_2^T\mathbf{Q}\mathbf{x}_2 \geq 0 \\ &\quad \Rightarrow \mathbf{x}_1^T\mathbf{Q}\mathbf{x}_1 - \mathbf{x}_2^T\mathbf{Q}\mathbf{x}_2 \geq 2\mathbf{x}_1^T\mathbf{Q}\mathbf{x}_2 - 2\mathbf{x}_2^T\mathbf{Q}\mathbf{x}_2 = (\mathbf{x}_1 - \mathbf{x}_2)^T 2\mathbf{Q}\mathbf{x}_2.\end{aligned}$$

From Theorem A.1.3, since $\mathbf{Q}$ is symmetric,

$$f(\mathbf{x}) = \mathbf{x}^T\mathbf{Q}\mathbf{x} \quad \Rightarrow \quad (\nabla f(\mathbf{x}))^T = 2\mathbf{Q}\mathbf{x}$$

Using this in the RHS of the previous inequality gives

$$f(\mathbf{x}_1) - f(\mathbf{x}_2) \geq (\mathbf{x}_1 - \mathbf{x}_2)^T(\nabla f(\mathbf{x}_2))^T \qquad \forall \mathbf{x}_1, \mathbf{x}_2$$

which proves convexity. □

A.4 Covariance Matrices and Matrix Products

A.4.1 Positive Semidefiniteness of Covariance Matrices

Theorem A.4.1. *Covariance matrices $\mathbf{C}_\mathbf{X}$ are always positive semi-definite.*

Proof. For an arbitrary direction $\mathbf{u}$, consider the scalar measurements $\mathbf{u}^T\mathbf{x}_i$. Their variance is

$$\begin{aligned}\sigma^2(\mathbf{u}) &= \frac{1}{n}\sum_{i=1}^{n}\left[\mathbf{u}^T(\mathbf{x}_i - \overline{\mathbf{x}})\right]^2 = \frac{1}{n}\sum_{i=1}^{n}\left[\mathbf{u}^T(\mathbf{x}_i - \overline{\mathbf{x}})\right]\left[\mathbf{u}^T(\mathbf{x}_i - \overline{\mathbf{x}})\right]^T = \\ &= \frac{1}{n}\sum_{i=1}^{n}\mathbf{u}^T(\mathbf{x}_i - \overline{\mathbf{x}})(\mathbf{x}_i - \overline{\mathbf{x}})^T\mathbf{u} = \mathbf{u}^T\left[\frac{1}{n}\sum_{i=1}^{n}(\mathbf{x}_i - \overline{\mathbf{x}})(\mathbf{x}_i - \overline{\mathbf{x}})^T\right]\mathbf{u} = \mathbf{u}^T\mathbf{C}_\mathbf{X}\mathbf{u}.\end{aligned}$$

Variance is non-negative, so

$$\mathbf{u}^T\mathbf{C}_\mathbf{X}\mathbf{u} \geq 0, \quad \forall \mathbf{u}.$$

Since $\mathbf{C}_\mathbf{X}$ is also symmetric, it is positive semi-definite. □

For centered data $\mathbf{X} \in \mathbb{R}^{m\times n}$, the covariance matrix is

$$\mathbf{C}_\mathbf{X} = \frac{1}{n}\mathbf{X}\mathbf{X}^T.$$

Its eigenvalues are non-negative and represent the variances along its unit eigenvectors.

A.4.2 Symmetry and Positive definiteness of Matrix Products

More generally, for any real matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$, both $\mathbf{A}\mathbf{A}^T$ and $\mathbf{A}^T\mathbf{A}$ are symmetric and positive semi-definite. Indeed,

$$(\mathbf{A}\mathbf{A}^T)^T = \mathbf{A}\mathbf{A}^T, \qquad (\mathbf{A}^T\mathbf{A})^T = \mathbf{A}^T\mathbf{A},$$

and

$$\begin{aligned}\mathbf{u}^T\mathbf{A}\mathbf{A}^T\mathbf{u} &= \left\|\mathbf{A}^T\mathbf{u}\right\|_2^2 \geq 0, \qquad \forall \mathbf{u} \in \mathbb{R}^m \\ \mathbf{x}^T\mathbf{A}^T\mathbf{A}\mathbf{x} &= \|\mathbf{A}\mathbf{x}\|_2^2 \geq 0, \qquad \forall \mathbf{x} \in \mathbb{R}^n\end{aligned}$$

In particular, these properties apply both to $\mathbf{X}\mathbf{X}^T \in \mathbb{R}^{m\times m}$ and $\mathbf{X}^T\mathbf{X} \in \mathbb{R}^{n\times n}$.

A.4.3 Null Spaces and Rank

Theorem A.4.2. *Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{m\times p}$ be two matrices and $\mathbf{x} \in \mathbb{R}^{n\times 1}$ be a column vector. Then, the following relations hold:*

$$\mathbf{A}\mathbf{x} = \mathbf{0} \quad \Leftrightarrow \quad \mathbf{A}^T\mathbf{A}\mathbf{x} = \mathbf{0} \tag{A.1}$$

$$\mathbf{A}\mathbf{B} = \mathbf{0} \quad \Leftrightarrow \quad \mathbf{A}^T\mathbf{A}\mathbf{B} = \mathbf{0}. \tag{A.2}$$

Proof. For Equation A.1, we have that

$$\mathbf{A}\mathbf{x} = \mathbf{0} \quad \Rightarrow \quad \mathbf{A}^T\mathbf{A}\mathbf{x} = \mathbf{0}.$$

Conversely, if $\mathbf{A}^T\mathbf{A}\mathbf{x} = \mathbf{0}$, we also have

$$(\mathbf{A}\mathbf{x})^T(\mathbf{A}\mathbf{x}) = \mathbf{x}^T\mathbf{A}^T\mathbf{A}\mathbf{x} = \mathbf{0}.$$

The last equality is a square norm, which implies $\mathbf{A}\mathbf{x} = \mathbf{0}$. Equation A.2 follows by applying Equation A.1 to each column of $\mathbf{B}$. □

The first relation shows that $\mathbf{A}$ and $\mathbf{A}^T\mathbf{A}$ have the same null space. Since they also have the same number of columns, the rank-nullity theorem gives

$$\operatorname{rank}(\mathbf{A}) = \operatorname{rank}(\mathbf{A}^T\mathbf{A}).$$

Applying the same argument to $\mathbf{A}^T$, and recalling that a matrix and its transpose have the same rank, we obtain

$$\operatorname{rank}(\mathbf{A}) = \operatorname{rank}(\mathbf{A}^T) = \operatorname{rank}(\mathbf{A}^T\mathbf{A}) = \operatorname{rank}(\mathbf{A}\mathbf{A}^T).$$

A.5 Singular Value Decomposition Theorem

Theorem A.5.1. *Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ be a matrix with rank $r = \operatorname{rank}(\mathbf{A}) > 0$. Then, there exist:*

i) a matrix $\mathbf{U} \in \mathbb{R}^{m\times r}$ with orthonormal columns,

ii) a matrix $\mathbf{V} \in \mathbb{R}^{n\times r}$ with orthonormal columns,

iii) a diagonal matrix $\mathbf{\Sigma} \in \mathbb{R}^{r\times r}$ with positive diagonal elements,

such that

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}^{1/2}\mathbf{V}^T, \qquad \mathbf{U}^T\mathbf{U} = \mathbf{I}_r, \qquad \mathbf{V}^T\mathbf{V} = \mathbf{I}_r.$$

In this decomposition:

- $\boldsymbol{\Sigma}$ *contains the* r *non-zero eigenvalues shared by* $\mathbf{A}\mathbf{A}^T$ *and* $\mathbf{A}^T\mathbf{A}$*,*
- *the columns of* $\mathbf{U}$ *are orthonormal eigenvectors of* $\mathbf{A}\mathbf{A}^T$ *associated with these eigenvalues,*
- *the columns of* $\mathbf{V}$ *are orthonormal eigenvectors of* $\mathbf{A}^T\mathbf{A}$ *associated with the same eigenvalues.*

Proof. First, notice that $\mathbf{A}\mathbf{A}^T$ is a real symmetric and positive semi-definite matrix of size $m \times m$. Moreover,

$$\operatorname{rank}(\mathbf{A}) = \operatorname{rank}(\mathbf{A}^T) = \operatorname{rank}(\mathbf{A}^T\mathbf{A}) = \operatorname{rank}(\mathbf{A}\mathbf{A}^T) = r.$$

From Theorem A.2.3, its eigenvalues are non-negative. Since its rank is r, exactly r eigenvalues are positive and the remaining $m - r$ are zero.

From Theorem 1.5.1, we can write

$$(\mathbf{A}\mathbf{A}^T)\mathbf{E} = \mathbf{E}\mathbf{D}.$$

where $\mathbf{E}$ is an orthogonal matrix containing the eigenvectors as columns. We order them so that

$$\mathbf{D} = \begin{bmatrix} \boldsymbol{\Sigma} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}, \qquad \boldsymbol{\Sigma} = \operatorname{diag}(\lambda_1, \ldots, \lambda_r), \quad \lambda_i > 0,$$

or, explicitly,

$$\mathbf{D} = \begin{bmatrix} \lambda_1 & 0 & \ldots & 0 & 0 \\ 0 & \ddots & \ldots & 0 & 0 \\ 0 & 0 & \lambda_r & \ldots & 0 \\ 0 & 0 & 0 & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & 0 \end{bmatrix} = \operatorname{diag}(\lambda_1, \ldots, \lambda_r, \underbrace{0, \ldots, 0}_{m-r}).$$

We then separate the first r columns of $\mathbf{E}$, associated with the positive eigenvalues, from the remaining $m - r$ columns:

$$\mathbf{E} \triangleq [\mathbf{U} \mid \mathbf{U}_*], \quad \mathbf{U} \in \mathbb{R}^{m \times r}, \quad \mathbf{U}_* \in \mathbb{R}^{m \times (m-r)}.$$

Due to the block structure of $\mathbf{D}$, we have

$$(\mathbf{A}\mathbf{A}^T)\,[\mathbf{U} \mid \mathbf{U}_*] = [\mathbf{U} \mid \mathbf{U}_*]\,\mathbf{D} = [\mathbf{U}\boldsymbol{\Sigma} \mid \mathbf{0}]\,.$$

Thus,

$$\mathbf{A}\mathbf{A}^T\mathbf{U} = \mathbf{U}\boldsymbol{\Sigma}, \qquad \mathbf{A}\mathbf{A}^T\mathbf{U}_* = \mathbf{0}. \tag{A.3}$$

Since $\mathbf{E}$ is orthogonal, its columns are orthonormal, so $\mathbf{U}^T\mathbf{U} = \mathbf{I}_r$. We also have

$$\mathbf{E}\mathbf{E}^T = [\mathbf{U} \mid \mathbf{U}_*]\,[\mathbf{U} \mid \mathbf{U}_*]^T = \mathbf{U}\mathbf{U}^T + \mathbf{U}_*\mathbf{U}_*^T = \mathbf{I}_m.$$

Applying Theorem A.4.2 to $\mathbf{A}^T$ and $\mathbf{U}_*$ inthe second expression of Equation A.3 implies

$$\mathbf{A}^T\mathbf{U}_* = \mathbf{0} \quad \Rightarrow \quad \mathbf{U}_*^T\mathbf{A} = \mathbf{0}.$$

Now, we define

$$\mathbf{V} = \mathbf{A}^T\mathbf{U}\boldsymbol{\Sigma}^{-1/2}. \tag{A.4}$$

The inverse square root is well-defined because all diagonal elements of $\boldsymbol{\Sigma}$ are positive. Then, we obtain

$$\begin{aligned}\mathbf{A}^T\mathbf{A}\mathbf{V} &\overset{(A.4)}{=} \mathbf{A}^T\mathbf{A}\mathbf{A}^T\mathbf{U}\boldsymbol{\Sigma}^{-1/2} \overset{(A.3)}{=} \mathbf{A}^T\mathbf{U}\boldsymbol{\Sigma}\boldsymbol{\Sigma}^{-1/2} = \\ &= \mathbf{A}^T\mathbf{U}\boldsymbol{\Sigma}^{1/2} = \mathbf{V}\boldsymbol{\Sigma}.\end{aligned}$$

Therefore, the columns of $\mathbf{V}$ are eigenvectors of $\mathbf{A}^T\mathbf{A}$ associated with the eigenvalues in $\boldsymbol{\Sigma}$.

Moreover, these columns are orthonormal

$$\mathbf{V}^T\mathbf{V} = \boldsymbol{\Sigma}^{-1/2}\mathbf{U}^T\mathbf{A}\mathbf{A}^T\mathbf{U}\boldsymbol{\Sigma}^{-1/2} = \mathbf{I}_r. \tag{A.5}$$

We can now combine these results to reconstruct $\mathbf{A}$

$$\begin{aligned}\mathbf{A} &= \overbrace{(\mathbf{U}\mathbf{U}^T + \mathbf{U}_*\mathbf{U}_*^T)}^{\mathbf{I}_m} \overset{(A.4.2)}{=} \mathbf{U}\overbrace{\boldsymbol{\Sigma}^{1/2}\boldsymbol{\Sigma}^{-1/2}}^{\mathbf{I}_r}\mathbf{U}^T\mathbf{A} = \\ &= \mathbf{U}\boldsymbol{\Sigma}^{1/2}\underbrace{(\mathbf{A}^T\mathbf{U}\boldsymbol{\Sigma}^{-1/2})^T}_{\mathbf{V}^T} = \mathbf{U}\boldsymbol{\Sigma}^{1/2}\mathbf{V}^T.\end{aligned}$$

Hence, matrix $\mathbf{A}$ can be decomposed into

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}^{1/2}\mathbf{V}^T$$

Notice that, in this process, we have found:

$$(\mathbf{A}\mathbf{A}^T)\mathbf{U} = \mathbf{U}\boldsymbol{\Sigma} \tag{A.6}$$

$$(\mathbf{A}^T\mathbf{A})\mathbf{V} = \mathbf{V}\boldsymbol{\Sigma}. \tag{A.7}$$

These relations show that $\mathbf{A}\mathbf{A}^T$ and $\mathbf{A}^T\mathbf{A}$ share the same r non-zero eigenvalues. Their corresponding orthonormal eigenvectors are the columns of $\mathbf{U}$ and $\mathbf{V}$, respectively. The diagonal elements of $\boldsymbol{\Sigma}^{1/2}$ are the square roots of these eigenvalues and are called the **singular values** of $\mathbf{A}$. □

Bibliography

[1] Jonathon Shlens. A tutorial on principal component analysis. *CoRR*, abs/1404.1100, 2014.

[2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, hardcover edition, 2011.

[3] Matthew Turk and Alex Pentland. Eigenfaces for recognition. *Journal of Cognitive Neuroscience*, 3(1):71–86, 1991.

[4] Matteo Gadaleta and Michele Rossi. Idnet: Smartphone-based gait recognition with convolutional neural networks. *Pattern Recognition*, 74:25–37, 2018.